

## 4. Project Design Phase

### 4.1 Introduction

The design phase transforms the requirements into a structured system design that defines the components, data flow, and functionality of ComicCraft. The system follows a modular architecture consisting of a user interface, an application server, a database with image storage, and a Gemini AI integration module that communicate through well-defined APIs.

### 4.2 System Design

ComicCraft follows a client-server architecture. The user interacts with the frontend, which sends requests to the FastAPI backend through REST APIs. The backend validates requests, builds prompts that include the story idea, style, and character descriptions, calls the Google Gemini text model to produce a structured script, then calls the Gemini image model for each panel. Generated images are stored on the server and linked to the comic in the database. The Gemini API key is kept only on the server.

### 4.3 System Architecture

- **Presentation Layer:** login, registration, story input, script editor, character manager, panel viewer, bubble editor, layout preview, and gallery pages.
- **Application Layer:** Python and FastAPI handling authentication, request validation, prompt construction, image composition, and business logic.
- **AI Layer:** Google Gemini text model (story, script, character descriptions) and Gemini image model (panel artwork).
- **Data Layer:** SQLite or PostgreSQL storing users, comics, characters, and panels, plus file storage for generated images.

### 4.4 System Architecture Flow

User → Frontend → FastAPI Backend → Prompt Builder → Gemini Text Model → Script (JSON) → Gemini Image Model → Panel Images → Database/Storage → Bubble Editor & Layout → Export → User

### 4.5 Module Design

| Module               | Design Description                                                                                                                                                 |
|----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Authentication       | Registration, login with password hashing, JWT token issuing and verification, profile management, logout.                                                         |
| Story Input          | Collects idea, genre, tone, art style, panel count, and optional characters; validates and sanitizes input.                                                        |
| AI Script Generator  | Sends a structured prompt to Gemini and receives JSON containing title, synopsis, characters, and panels (scene, dialogue, caption, camera angle); allows editing. |
| Character Management | Stores character profiles and builds a consistent description block that is appended to every panel prompt; optional reference sheet generation.                   |

| Module                 | Design Description                                                                                                                                     |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| Panel Image Generation | Builds a panel prompt from scene, style, and character blocks; calls the Gemini image model; saves the image; supports regeneration of a single panel. |
| Speech Bubble Editor   | Overlays dialogue and captions as editable bubbles (position, size, style) on top of text-free artwork.                                                |
| Layout and Export      | Arranges panels into pages using layout templates and exports PDF or PNG using Pillow and ReportLab.                                                   |
| Gallery and History    | Retrieves saved comics, allows opening, editing, duplicating, and deleting.                                                                            |

## 4.6 Database Design

| Table          | Main Fields                                                                                         |
|----------------|-----------------------------------------------------------------------------------------------------|
| users          | id, name, email, password_hash, default_style, created_at                                           |
| comics         | id, user_id, title, idea, genre, art_style, synopsis, status, created_at                            |
| characters     | id, comic_id, name, appearance, clothing, personality, reference_image_path                         |
| panels         | id, comic_id, panel_no, scene_description, dialogue, caption, camera_angle, image_path, bubble_json |
| generation_log | id, user_id, comic_id, type (script/image), prompt, status, created_at                              |

## 4.7 API Design

| Method             | Endpoint                     | Purpose                                                  |
|--------------------|------------------------------|----------------------------------------------------------|
| POST               | /auth/register               | Create a new user account                                |
| POST               | /auth/login                  | Authenticate and return JWT                              |
| POST               | /comics                      | Create a comic from a story idea and generate the script |
| GET                | /comics                      | List the user's saved comics                             |
| GET / PUT / DELETE | /comics/{id}                 | View, edit, or delete a comic                            |
| POST               | /comics/{id}/characters      | Add or update a character profile                        |
| POST               | /comics/{id}/panels/generate | Generate images for all panels                           |
| POST               | /panels/{id}/regenerate      | Regenerate a single panel                                |
| PUT                | /panels/{id}/bubbles         | Save speech bubble and caption layout                    |
| GET                | /comics/{id}/export          | Export the comic as PDF or PNG                           |

## 4.8 Security Design

- Passwords are stored as hashes, never as plain text.
- JWT tokens protect all private endpoints, and each query is filtered by user\_id.
- The Gemini API key is stored in environment variables on the server.
- User input is validated and sanitized before being sent to the AI model.

- Gemini safety settings and output checks are used to block unsafe content.
- Uploaded and generated files are stored with unique names and served only to their owner.